Assignment-7

Problem Statement:
Implement stack as ADT and apply it for different expression conversions (infix to postfix or infix to prefix (Any one), prefix to postfix or prefix to infix, postfix to infix or postfix to prefix (Any one)).

Objective:
1. To study Stack and its operations.
2. To study the importance of expression conversions

Theory
Stack: Special case of ordered lists also called as restricted / controlled list where insertion and deletion happen at only one end called as top of stack (homogeneous collection of elements) Elements are added to and removed from the top of the stack is the first to be removed (LIFO: Last IN, First Out). Only access to the stack is the top element

Basic Stack Operations
- isEmpty() - Checking stack is empty
- isFull() - Checking stack is full
- push() - Pushing Element on top of stack
- pop() - Popping top element from stack

Applications of Stack

1. Expression evaluation (Postfix and Prefix)
2. Infix to Postfix and Prefix conversion
3. Function call management (Call Stack)
4. Recursion
5. Parenthesis matching

6 Undo/Redo operations in editors
7 Browser back and forward navigation
8 Depth First Search (DFS) in graphs
9 Syntax parsing in compilers
10 String reversal.

Expression conversion is the process of converting an arithmetic from one notation to another using a stack. The three common notations are:

Infix: Operator is between operands (eg. A+B)
Prefix (Polish): Operator is before operands (eg. +AB)
Postfix (Reverse Polish): Operator is after operands (eg AB+)

A stack is used to temporarily store operators and parenthesis during the conversion.

Implementation:

- Platform:
  - 64-bit Open Source Linux or its derivatives.
  - Open Source C Programming tool like gcc/Eclipse Editor
- Test Conditions:

1) Input infix expressions and check the postfix or prefix output
2) Input postfix or prefix expression and check its infix, prefix or postix.

Pseudo Code

Push

Push ()

```
Algorithm push (stack, top, size, element):
    if top >= size - 1 then
        Print "Stack Overflow"
        Return
    end if
    top = top + 1
    stack[top] = element
End Algorithm
```

Pop ()

```
Algorithm pop (stack, pop):
    if top < 0 then
        Print "Stack Underflow"
        Return -1
    end if
    element = stack[top]
    top = top - 1
    Return element
End Algorithm
```

Infix to Postfix

```
infix to postfix (exp):
    for each, char 'ch' in exp:
        if 'ch' is operand: appad to output
        else if 'ch' == '(': push(ch)
        else if 'ch' = = ')':
            while stak top != '(': append pop()
            pop() // remove '('
        else:
            while stack not empty and preceedence(ch)
            <= precedence (top):
                ~~push(ch)~~
                ~~Append pop()~~
```

push(ch)
while stack not empty : append pop():

Prefix to Infix Conversion

```
prefixToInfix(exp):
    for 'ch' in exp from right to left:
        if 'ch' is operand : push(ch)
        else:
            op1 = pop(), op2 = pop()
            push("(" + op1 + ch + op2 + ")")
    return pop()
```

Prefix to Postfix

```
prefixToPostfix(exp):
    for 'ch' in exp from right to left:
        if 'ch' is operand : push(ch)
        else:
            op1 = pop(), op2 = pop()
            push(op1 + op2 + ch)
    return pop()
```

Postfix to Infix Conversion

```
postfixToInfix(exp):
    for 'ch' in exp:
        if 'ch' is operand : push(ch)
        else:
            op2 = pop(), op1 = pop()
            push("(" + op1 + ch + op2 + ")")
    return pop()
```

Postfix to Prefix Conversion

```
postfixtoPrefix(exp):
    for 'ch' in exp:
        if 'ch' is operand: push(ch)
        else:
            op2 = pop(), op1 = pop()
            push(ch + op1 + op2)
    return pop()
```

Time Complexity

Push : O(1) - ~~Direct insertion~~
Pop : O(1)
Infix to Postfix : O(N)
Prefix to Infix / Postfix : O(N)
Postfix to Infix / Prefix : O(N)

FAQs

~~1) Isha~~

1) What is the advantage of prefix and postfix over infix expressions?
- No Parenthesis : Eliminates the need for grouping parenthesis
- No Precedence Rules : Allows straightforward sequential evaluation without checking operator precedence.

2) Why is stack used in expression conversion?
The Last in First out (LIFO) property naturally handles operator reversal, nested parenthesis and precedence tracking.

3) Give stack full and stack empty conditions for stack.
- Stack Empty : top == -1
- Stack Full : top == MAX - 1